

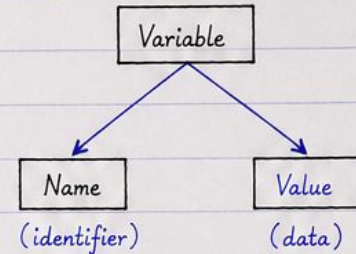
LANGUAGE CONSTRUCTS

1. Variables :

→ Variable एक नामित storage location होता है जिसका उपयोग data को store करने के लिए किया जाता है।

→ Program के execution के दौरान variable का value बदल सकता है।

→ हर variable का एक specific data type होता है जो यह निर्धारित करता है कि उसमें किस प्रकार का data store होगा। * Value change हो सकता है।



2. Types of Variables (Data Types):

Java में data types को दो भागों में बाँटा गया है -

(1) Primitive (Basic)

→ ये predefined होते हैं और इनका size fixed होता है।

→ 8 types :

Type	Size	Description	Example
byte	1 byte	छोटी integer value	byte b = 10;
short	2 bytes	छोटी integer value	short s = 1000;
int	4 bytes	Integer value	int n = 25;
long	8 bytes	बड़ी integer value	long l = 123456L;
float	4 bytes	Floating point value	float f = 12.5f;
double	8 bytes	Double precision value	double d = 12.345;
char	2 bytes	Character value	char ch = 'A';
boolean	1 bit	True / False value	boolean flag = true;

(2) Non-Primitive (Reference)

→ ये user-defined या predefined classes होते हैं और इनका size fixed नहीं होता।

→ Types :

- String
- Array
- Class
- Interface
- Object आदि।

Example :

```
String name = "Braj Kant";  
int arr[] = {1, 2, 3};  
MyClass obj = new MyClass();
```

3. Type Declarations :

→ Variable को use करने से पहले उसे declare करना जरूरी होता है।

`dataType variableName;` ← Declaration

→ एक साथ कई variables declare किए जा सकते हैं।

`dataType var1, var2, var3;`

→ Declaration के साथ value assign करना (Initialization):

`dataType variableName = value;`

→ Example :

```
int a;           // single variable declaration  
int b, c;        // multiple variables declaration  
int a = 10;      // declaration with initialization  
float f = 5.5f;  // float variable initialization
```

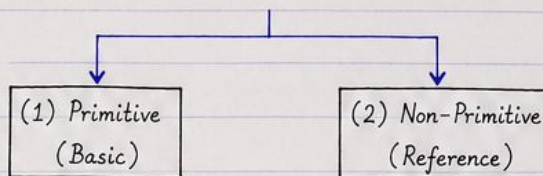
Points to Remember :

- Variable का नाम meaningful रखना चाहिए।
- Variable name case-sensitive होता है। (age और Age अलग हैं)
- Variable name letter, digit, _ या \$ से शुरू हो सकता है, digit से नहीं।
- Java में reserved words को variable name के रूप में use नहीं कर सकते।

DATA TYPES : INTEGER, FLOATING POINT TYPE, CHARACTER

→ Java एक Strongly-typed language है, इसलिए हर variable को declare करते समय उसका data type specify करना होता है।

→ Data types को दो भागों में बाँटा गया है -



1. Integer Type :

→ Integer type उन numbers को store करता है जिनमें कोई decimal point नहीं होता।

→ Java में अलग-अलग size के integer types होते हैं।

→ Size बढ़ने के साथ range भी बढ़ जाती है।

Type	Size	Range	Default Value	Example
byte	1 byte	-128 to 127	0	byte b = 10;
short	2 bytes	-32,768 to 32,767	0	short s = 1000;
int	4 bytes	-2^{31} to $2^{31}-1$ (-2147483648 to 2147483647)	0	int n = 25;
long	8 bytes	-2^{63} to $2^{63}-1$ (very large range)	0L	long l = 123456L;

Example :

```
int a = 10;           // decimal
int b = -20;          // negative integer
int c = 075;          // octal (61 in decimal)
int d = 0x2A;         // hexadecimal (42 in decimal)
long big = 1234567890123L; // long integer
```

2. Floating Point Type :

→ Floating point type उन numbers को store करता है जिनमें decimal point होता है।

→ यह real numbers (fractional part) को represent करने के लिए उपयोग होता है।

→ Java में मुख्य रूप से दो floating point types होते हैं -

Type	Size	Range (approx.)	Default Value	Example
float	4 bytes	1.4E-45 to 3.4028235E+38	0.0f	float f = 5.5f;
double	8 bytes	4.9E-324 to 1.797693E+308	0.0d	double d = 3.14159;

Example :

```
float f1 = 3.14f;      // float literal
float f2 = -0.25f;     // negative float
double d1 = 3.1415926535; // double literal
double d2 = -123.456e2; // scientific notation (-123.456 × 102)
```

3. Character Type :

→ Character type single character को store करता है।

→ Java में character को single quote (' ') के अंदर लिखा जाता है।

→ यह Unicode value के रूप में store होता है।

→ Size हमेशा 2 bytes (16 bits) होता है।

Type	Details
char	- Size : 2 bytes - Range : '\u0000' (0) to '\uFFFF' (65535) - Default Value : '\u0000' - Example : char ch = 'A';

Example :

```
char ch1 = 'A';      // uppercase letter
char ch2 = 'z';      // lowercase letter
char ch3 = '7';      // digit
char ch4 = '\n';     // escape character (new line)
```

Note :

→ 'A' और 'a' अलग-अलग characters हैं।

→ 'A' valid है, पर 'AB' invalid है (क्योंकि char एक ही character store करता है)।

→ Unicode value प्राप्त करने के लिए type casting का use करते हैं, जैसे - int i = (int) 'A'; // 65

1. Boolean Type :

- Boolean type logical values को store करता है।
- इसमें केवल दो values होती हैं - true और false.
- इसका size JVM पर depend करता है, लेकिन आम तौर पर 1 byte (8 bits) माना जाता है।
- Boolean का उपयोग conditions, control statements और decision making के लिए किया जाता है।

Details :

Type	boolean
Values	true, false
Size	dependent on JVM (typically 1 byte)
Default Value	false
Example	boolean flag = true;

Example :

```
boolean a = true;
boolean b = false;
if (a) {
    System.out.println("Condition is true");
}
else {
    System.out.println("Condition is false");
}
```

Note :

Boolean values का सीधा numeric conversion नहीं होता।

2. All Operators in Java :

- Operators का उपयोग variables और values पर operations करने के लिए किया जाता है।
- Java में Operators को निम्न प्रकार से classify किया गया है -

(1) Arithmetic Operators

Operator	Meaning	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Modulus (Remainder)	a % b
++	Increment	a ++
--	Decrement	a --

(2) Relational Operators

Operator	Meaning	Example
==	Equal to	a == b
!=	Not equal to	a != b
>	Greater than	a > b
<	Less than	a < b
>=	Greater than or equal to	a >= b
<=	Less than or equal to	a <= b

(3) Logical Operators

Operator	Meaning	Example
&&	Logical AND	(a > 5 && b < 10)
	Logical OR	(a > 5 b < 10)
!	Logical NOT	!(a > b)

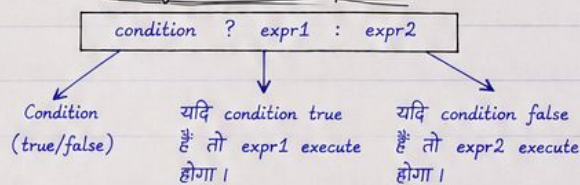
(4) Assignment Operators

Operator	Meaning	Example
=	Assign	a = 10
+=	Add and assign	a += 5 (a = a + 5)
-=	Subtract and assign	a -= 5 (a = a - 5)
*=	Multiply and assign	a *= 5 (a = a * 5)
/=	Divide and assign	a /= 5 (a = a / 5)
%=	Modulus and assign	a %= 5 (a = a % 5)

(5) Bitwise Operators

Operator	Meaning	Example
&	Bitwise AND	a & b
	Bitwise OR	a b
^	Bitwise XOR	a ^ b
~	Bitwise Complement	~a
<<	Left Shift	a << 2
>>	Right Shift	a >> 2
>>>	Unsigned Right Shift	a >>> 2

(6) Ternary (Conditional) Operators



Example :

```
int max = (a > b) ? a : b;
यदि a > b true हैं तो max = a
// अन्यथा max = b
```

(7) Type Operators

Operator	Meaning	Example
instanceof	Checks object type	obj instanceof String
(type)	Type Casting	int a = (int) 5.6;

(8) Other Operators

Operator	Meaning	Example
.	Member Access	System.out.println();
[]	Array Access	arr[0]
()	Method Call	method();
new	Object Creation	new ClassName();

Note : Operator precedence होती है, जैसे * , / , % > + , - > Relational > Logical AND (&&) > Logical OR (||) > Assignment (=)

→ Precedence के कारण कुछ operations पहले execute होते हैं।

1. Iteration (Looping) Statements :

→ Iteration का मतलब किसी block of code को बार-बार execute करना होता है जब तक कोई condition true रहती है।

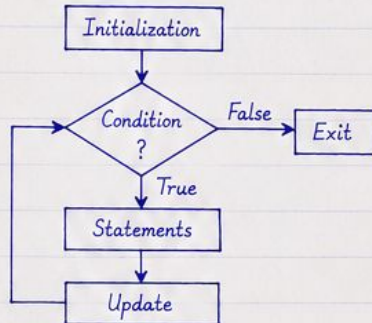
→ Java में मुख्यतः तीन प्रकार के loops होते हैं -

(i) for loop (ii) while loop (iii) do-while loop

(i) for loop :

```
for(initialization; condition; update) {  
    // statements  
}
```

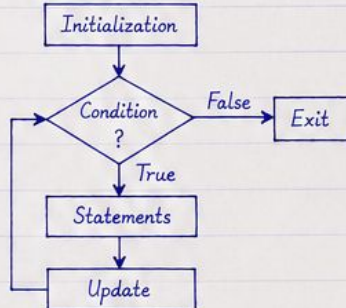
Flowchart :



(ii) while loop :

```
while(condition) {  
    // statements  
    update;  
}
```

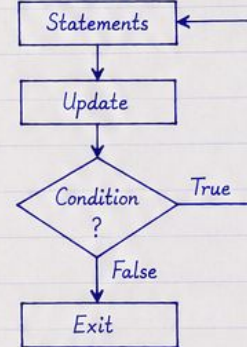
Flowchart :



(iii) do-while loop :

```
do {  
    // statements  
    update;  
} while(condition);
```

Flowchart :



Note : for loop में entry से पहले condition check होती है, जबकि do-while loop में statements कम से कम एक बार execute होते हैं क्योंकि condition बाद में check होती है।

2. Jump Statements :

→ Jump statements का उपयोग program की normal flow को एक स्थान से दूसरे स्थान पर भेजने के लिए किया जाता है।

Statement	Description
break	Loop या switch से बाहर निकलने के लिए।
continue	Loop के current iteration को skip करके अगले iteration पर जाने के लिए।
return	Method से control वापस caller को भेजने के लिए।
goto (Not in Java)	किसी label पर jump करने के लिए (Java में नहीं)।

Example :

```
for(int i=1; i<=5; i++) {  
    if(i == 3) continue; // 3 skip होगा  
    if(i == 5) break; // 5 पर loop terminate  
    System.out.print(i + " ");  
}  
Output: 1 2 4
```

3. if-then-else Clause :

→ if-then-else का उपयोग decision making के लिए किया जाता है।

→ Condition true या false के आधार पर different statements execute होते हैं।

(i) if statement :

```
if(condition) {  
    // statements  
}
```

(ii) if-else statement :

```
if(condition) {  
    // statements (true)  
} else {  
    // statements (false)  
}
```

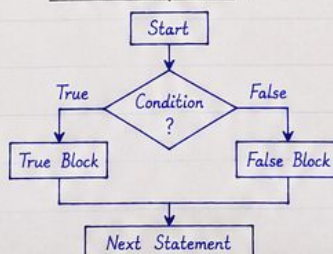
(iii) if-else-if ladder :

```
if(condition1) {  
    // statements  
} else if(condition2) {  
    // statements  
} else if(condition3) {  
    // statements  
} else {  
    // statements  
}
```

(iv) Nested if statement :

```
if(condition1) {  
    if(condition2) {  
        // statements  
    } else {  
        // statements  
    }  
} else {  
    // statements  
}
```

Flowchart (if-else) :



Note :

- ✓ Condition का result boolean (true/false) होना चाहिए।
- ✓ Multiple conditions के लिए if-else-if ladder का उपयोग करें।
- ✓ Nested if का उपयोग complex decisions के लिए किया जाता है।

1. Conditional Expressions :

→ Conditional expression एक expression होता है जो condition के आधार पर एक value return करता है।

→ इसे ternary operator ($? :$) के द्वारा लिखा जाता है।

Syntax : condition ? expr1 : expr2

Condition	If True	If False
-----------	---------	----------

→ यदि condition true होती है तो expr1 execute होकर value return करता है, अन्यथा expr2 execute होकर value return करता है।

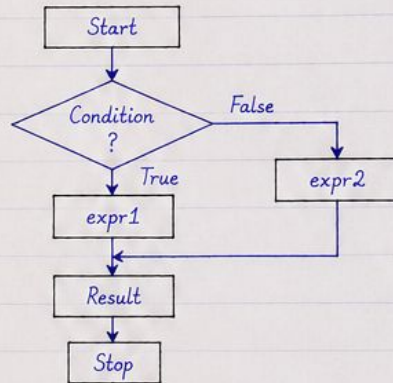
Example :

```
int a = 10, b = 20, max;  
max = (a > b) ? a : b;    // max = 20  
System.out.println("Maximum = " + max);
```

Note :

(i) Result का data type expr1 और expr2 दोनों का same होना चाहिए।

(ii) Conditional expressions को assignment के right side में use किया जाता है।



More Example :

```
int x = (a > b) ? a - b : b - a;
char grade = (marks >= 33) ? 'P' : 'F';
```

2. Input using Scanner Class and Output Statement :

(A) Input using Scanner Class :

→ Scanner class java.util package में होती है।

→ इस class के द्वारा user से input प्राप्त किया जाता है।

Steps :

1. Scanner class का object बनाते हैं।

2. *Different methods* के द्वारा *input read* करते हैं।

Example :

```
import java.util. Scanner;

class InputExample {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in); // Scanner object
        System.out.print("Enter your name: ");
        String name = sc.nextLine();
        System.out.print("Enter age: ");
        int age = sc.nextInt();
        System.out.print("Enter marks: ");
        float marks = sc.nextFloat();
        System.out.println("Name = " + name);
        System.out.println("Age = " + age);
        System.out.println("Marks = " + marks);
        sc.close(); // Close scanner
    }
}
```

Scanner Class के Important Methods :

Method	Return Type	Description
nextInt()	int	Integer input के लिए
nextFloat()	float	Float input के लिए
nextDouble()	double	Double input के लिए
nextLine()	String	पूरी line (String) input के लिए
next()	String	एक word (space तक) input के लिए
nextBoolean()	boolean	Boolean input के लिए
nextChar()	char	Character input के लिए

(B) Output Statement :

→ Java में output display करने के लिए System.out का use होता है।

Output Methods	Description
System.out.print()	Output देता है बिना next line के
System.out.println()	Output देता है और next line पर जाता है
System.out.printf()	Formatted output देता है
System.out.flush()	Buffer में बचा output तुरंत भेजता है

Example :

```
int a = 10, b = 20;
System.out.print("Sum = ");           // Same line पर output
System.out.println(a + b);            // Next line पर output
System.out.printf("a = %d, b = %d", a, b);
System.out.println();
System.out.println("Thank You!");
```

Note : • System.in standard input stream है, जिससे user का data प्राप्त होता है।

- `nextInt()`, `nextFloat()` आदि token (space तक) पढ़ते हैं, जबकि `nextLine()` पूरी line पढ़ता है।

- Always close the Scanner using `sc.close()`, ताकि resources free हो सकें।

1. Loops :

- Loop का उपयोग किसी block of code को बार-बार execute करने के लिए किया जाता है।
- Java में तीन प्रकार के Loops होते हैं -
(i) for loop (ii) while loop (iii) do-while loop

Loop	Entry Control	Condition Check	Execute कम से कम एक बार
for	Yes	पहले	No
while	Yes	पहले	No
do-while	No	बाद में	Yes

(i) for loop :

```
for (int i = 1; i <= n; i++) {
    // statements
}
```

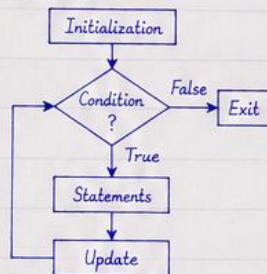
(ii) while loop :

```
while (condition) {
    // statements
}
```

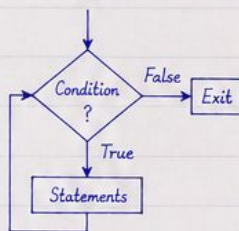
(iii) do-while loop :

```
do {
    // statements
} while (condition);
```

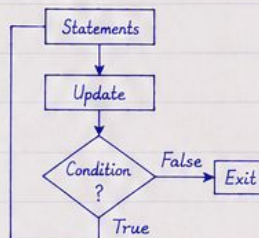
Flowchart :



Flowchart :



Flowchart :



Example :

```
// for loop
for (int i = 1; i <= 5; i++)
    System.out.print(i + " ");
```

```
// while loop
int i = 1;
while (i <= 5) {
    System.out.print(i + " ");
    i++;
}
```

```
// do-while loop
int i = 1;
do {
    System.out.print(i + " ");
    i++;
} while (i <= 5);
```

2. Switch Case :

- Switch case का उपयोग एक variable के different values के आधार पर अलग-अलग statements execute करने के लिए किया जाता है।
- यह if-else ladder का बेहतर और आसान विकल्प होता है।

```
switch (expression) {
    case value1:
        // statements
        break;
    case value2:
        // statements
        break;
    ...
    default:
        // statements
}
```

Example :

```
int day = 2;
switch (day) {
    case 1: System.out.println("Mon");
        break;
    case 2: System.out.println("Tue");
        break;
    case 3: System.out.println("Wed");
        break;
    default: System.out.println("Invalid");
}
```

3. Arrays :

- Array एक data structure होता है जिसमें same type के multiple elements को store किया जाता है।
- Array के elements index के द्वारा access किए जाते हैं।

Declaration :

```
type arrayName[] = new type[size];
या
type arrayName[] = {val1, val2, ...};
```

Example :

```
int marks[] = new int[5];
marks[0] = 85;
marks[1] = 90;
marks[2] = 78;
...
marks[4] = 88;
```

Array Diagram :

Index :	0	1	2	3	4
marks :	85	90	78	92	88

Traversing Array :

```
for (int i = 0; i < marks.length; i++) {
    System.out.println(marks[i]);
}
```

4. Methods :

- Method एक block of code होता है जिसे किसी specific task को perform करने के लिए define किया जाता है।
- यह code reusability को बढ़ाता है और program को modular बनाता है।

Method Declaration (Syntax) :

```
returnType methodName(parameter list) {
    // statements
    return value; // (if returnType is not void)
}
```

Example :

```
int add(int a, int b) {
    int sum = a + b;
    return sum;
}
// Calling the method
int result = add(10, 20);
System.out.println("Sum = " + result);
```

Types of Methods :

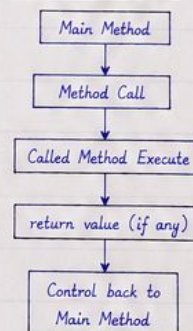
(i) Built-in Methods
जैसे: println(), nextInt(), length(), etc.

(ii) User Defined Methods
जो programmer स्वयं बनाता है।

Advantages :

- Code Reusability
- Modularity
- Easy to Maintain
- Reduces Code Length

Flow of Method Call :



Example (Void Method) :

```
void greet() {
    System.out.println("Hello Java!");
}
public static void main(String[] args) {
    greet(); // method call
}
```

Note :

Method call के time actual arguments pass होते हैं और method के अंदर formal parameters receive होते हैं।